IMPLEMENTATION REPORT

Front POS — Backend

Laravel API scaffold: architecture, schema, endpoints, and how it was built

---

Stack: Laravel 13 (PHP 8.3), MySQL (SQLite for interim local dev), Sanctum, Spatie Permission

Scope: Sprint 0 — Foundation (auth, terminal registration, shift/till open, shared domain schema, audit trail, tests)

Companion frontend: React + TypeScript (Vite), consuming this API over `/api/v1`

Source guide: Front POS End-to-End Workflows implementation guide

Contents

1. Overview & Tech Stack

The backend is a Laravel 13 JSON API living in `/backend` of the monorepo, built as the API half of a two-developer Front POS project (the other half being a React + TypeScript terminal UI in `/frontend`). It implements the **Sprint 0 — Foundation** milestone from the project's own delivery plan: authentication, terminal registration, shift/till opening, and the full shared data model for every domain in the spec — without yet implementing the sale/payment/kitchen/credit-note business logic that later sprints will add on top of this schema.

Concern	Choice	Why
Framework	Laravel 13 (PHP 8.3)	Chosen stack for the API side; latest stable release at scaffold time.
Auth	Laravel Sanctum	Lightweight token auth suited to a single-page/terminal client talking to one API.

Roles/permissions	spatie/laravel-permission	Battle-tested role & permission package; used for cashier/manager/admin roles and server-side threshold checks.
Database (target)	MySQL	Team's chosen production/shared DB.
Database (current local)	SQLite	Interim placeholder — MySQL install needs Windows admin elevation this session couldn't grant. One `.env` line switches it once MySQL is installed.
Money columns	`decimal(12,4)`	Fixed-precision decimals per the guide's mandatory control — avoids float rounding drift on tax/discount math.
Domain identifiers	UUID primary keys on Sale, Payment, CreditNote, KitchenOrder(+lines)	The guide requires these five aggregates to carry a unique, immutable identifier.
Timestamps	UTC	Mandatory control: "store timestamps in UTC".

2. Project Structure

```
SIMBISAPOS/
├── Front_POS_End_to_End_Workflows (1).pdf
├── backend/                          Laravel 13 API
│   ├── app/
│   │   ├── Enums/                    SaleStatus, PaymentStatus, CreditNoteStatus,
│   │   │                             KitchenOrderStatus, TableStatus, ShiftStatus,
│   │   │                             TillStatus, TerminalStatus
│   │   ├── Http/Controllers/Api/     AuthController, TerminalController,
│   │   │                             ShiftController, TillController
│   │   ├── Models/                   Store, Terminal, User, Shift, Till,
│   │   │                             CashMovement, Customer, Item, RestaurantTable,
│   │   │                             Sale, SaleLine, Payment, CreditNote,
│   │   │                             CreditNoteLine, KitchenOrder, KitchenOrderLine,
│   │   │                             AuditLog
│   │   └── Services/                 AuditLogger
│   ├── config/pos.php                POS-specific config (manager thresholds)
│   ├── database/
│   │   ├── migrations/               21 migrations — one per table
│   │   └── seeders/                  RolePermissionSeeder, DevDataSeeder
│   ├── routes/api.php                All endpoints, versioned under /api/v1
│   └── tests/Feature/PosFoundationTest.php
└── frontend/                         React + TypeScript (Vite) — separate report
```

3. Database Schema

21 tables were migrated, split into three groups: plain auto-increment reference/operational tables, UUID-keyed domain aggregates (the five the guide singles out for immutable identifiers, plus their line tables), and the audit trail.

3.1 Reference & operational tables

Table	Purpose	Key columns
`stores`	Store/site	name, code (unique), timezone
`terminals`	Registered POS terminals	store_id, registration_code (unique), status, app_version, last_seen_at
`users`	Staff — cashier/server/supervisor/manager	employee_code (unique), pin_hash, store_id, active
`shifts`	Cashier shift lifecycle	user_id, terminal_id, status, opened_at, closed_at
`tills`	Till/drawer session tied to a shift	shift_id, opening_float, closing_count, variance, manager_verified_by
`cash_movements`	Paid-in/out, cash drop, no-sale, till-open audit	till_id, type, amount, reason, approved_by
`customers`	Minimal customer profile	loyalty_id (unique), account_status, store_credit_balance
`items`	Minimal sellable-item reference (full product admin is out of scope)	sku (unique), price, tax_rate
`tables`	Restaurant floor tables	store_id, label, status, server_id, guest_count

3.2 UUID-keyed domain aggregates

Table	Purpose	Key columns
`sales`	Sale state machine	uuid pk, store/terminal/shift/customer/table/cashier ids, status, subtotal/discount/tax/total (decimal 12,4)
`sale_lines`	Basket lines	uuid pk, sale_id (uuid fk), item_id, quantity, unit_price, line_total, returned_quantity
`payments`	Payment state machine	uuid pk, sale_id, **idempotency_key (unique)**, tender_type, status, amount, provider_reference

`credit_notes`	Credit note state machine	uuid pk, original_sale_id, status, reason, refund_method, total, approved_by
`credit_note_lines`	Returned lines	uuid pk, credit_note_id, sale_line_id, quantity, amount, disposition
`kitchen_orders`	Kitchen order state machine	uuid pk, sale_id, table_id, status
`kitchen_order_lines`	Lines routed to kitchen stations	uuid pk, kitchen_order_id, item_id, station, status, is_duplicate

> The `payments.idempotency_key` unique constraint and the UUID primary keys directly implement two of the guide's mandatory controls: "a sale, payment, refund, credit note and kitchen line must each have a unique immutable identifier" and "all payment, sale, kitchen-send and refund requests must be idempotent."

3.3 Audit log

`backend/database/migrations/2026_09_01_111500_create_audit_logs_table.php`

```
Schema::create('audit_logs', function (Blueprint $table) {
    $table->id();
    $table->foreignId('actor_id')->nullable()->constrained('users')->nullOnDelete();
    $table->string('action');
    $table->string('auditable_type')->nullable();
    $table->string('auditable_id')->nullable();
    $table->json('before')->nullable();
    $table->json('after')->nullable();
    $table->string('reason')->nullable();
    $table->timestamps();

    $table->index(['auditable_type', 'auditable_id']);
});
```

A single polymorphic table records every discount, price override, void, refund, drawer open, and manager override — per the mandatory control that all of these "must be audited." It's populated through the `AuditLogger` service (Section 6), not written to directly by controllers.

3.4 Example migration — `sales`

`backend/database/migrations/2026_09_01_110800_create_sales_table.php`

```
Schema::create('sales', function (Blueprint $table) {
    $table->uuid('id')->primary();
    $table->foreignId('store_id')->constrained()->restrictOnDelete();
    $table->foreignId('terminal_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('shift_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('customer_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('table_id')->nullable()->constrained('tables')->nullOnDelete();
    $table->foreignId('cashier_id')->nullable()->constrained('users')->nullOnDelete();
    $table->string('status')->default('draft');
    $table->decimal('subtotal', 12, 4)->default(0);
    $table->decimal('discount_total', 12, 4)->default(0);
    $table->decimal('tax_total', 12, 4)->default(0);
    $table->decimal('total', 12, 4)->default(0);
    $table->timestamp('completed_at')->nullable();
    $table->timestamps();
});
```

4. State Machine Enums

Every state model from the guide's Section 14 table was transcribed directly into a PHP 8.1+ backed enum, so the normal path and every named exception state exist as first-class, typed values rather than free-text strings.

`backend/app/Enums/SaleStatus.php`

```
enum SaleStatus: string
{
    case Draft = 'draft';
    case Priced = 'priced';
    case PaymentPending = 'payment_pending';
    case Paid = 'paid';
    case Committed = 'committed';
    case Completed = 'completed';

    case Suspended = 'suspended';
    case Cancelled = 'cancelled';
    case PaymentUnknown = 'payment_unknown';
    case Failed = 'failed';
    case Voided = 'voided';
    case Returned = 'returned';
}
```

The same pattern was repeated for `PaymentStatus`, `CreditNoteStatus`, `KitchenOrderStatus`, `TableStatus`, and `ShiftStatus` — each enum's cases are copied verbatim from the guide's "Normal completion path" and "Exception states" columns. Two supporting enums (`TillStatus`, `TerminalStatus`) were added for entities the guide describes narratively but doesn't tabulate. These enums are wired onto their models via Eloquent's native enum casting (see Section 5), so `$sale->status` returns a real `SaleStatus` instance, not a string.

5. Eloquent Models

One model per table, using Laravel 13's attribute-based `#[Fillable( … )]` / `#[Hidden( … )]` configuration (the style the framework's own scaffolded `User` model already used) instead of the older `protected $fillable` array convention. UUID-keyed models use Laravel's built-in `HasUuids` concern.

backend/app/Models/User.php (edited from the framework default)

```
#[Fillable(['name', 'email', 'password', 'employee_code', 'pin_hash', 'store_id', 'active'])]
#[Hidden(['password', 'pin_hash', 'remember_token'])]
class User extends Authenticatable
{
    use HasApiTokens, HasFactory, HasRoles, Notifiable;

    protected function casts(): array
    {
        return [
            'email_verified_at' ⇒ 'datetime',
            'password' ⇒ 'hashed',
            'active' ⇒ 'boolean',
        ];
    }

    public function store(): BelongsTo
    {
        return $this→belongsTo(Store::class);
    }
}
```

`HasApiTokens` (Sanctum) and `HasRoles` (Spatie) are the two traits that turn a plain user into something that can authenticate over the API and carry roles/permissions.

backend/app/Models/Sale.php (excerpt)

```php
#[Fillable([
    'store_id', 'terminal_id', 'shift_id', 'customer_id', 'table_id', 'cashier_id',
    'status', 'subtotal', 'discount_total', 'tax_total', 'total', 'completed_at',
])]
class Sale extends Model
{
    use HasFactory, HasUuids;

    protected function casts(): array
    {
        return [
            'status' => SaleStatus::class,
            'subtotal' => 'decimal:4',
            'discount_total' => 'decimal:4',
            'tax_total' => 'decimal:4',
            'total' => 'decimal:4',
            'completed_at' => 'datetime',
        ];
    }

    public function lines(): HasMany { return $this->hasMany(SaleLine::class); }
    public function payments(): HasMany { return $this->hasMany(Payment::class); }
    public function creditNotes(): HasMany { return $this->hasMany(CreditNote::class, 'original
    public function kitchenOrders(): HasMany { return $this->hasMany(KitchenOrder::class); }
}
```

`HasUuids` makes Eloquent generate a UUID for `id` on creation instead of relying on auto-increment, and the enum cast on `status` means application code can write `$sale->status === SaleStatus::Completed` rather than comparing raw strings.

6. Services — Audit Logging

Rather than have every controller write to `audit_logs` directly, a small service centralises it so future sprints (discounts, voids, refunds, manager overrides) call one consistent method.

backend/app/Services/AuditLogger.php

```
class AuditLogger
{
    public function log(
        ?User $actor,
        string $action,
        ?Model $auditable = null,
        ?array $before = null,
        ?array $after = null,
        ?string $reason = null,
    ): AuditLog {
        return AuditLog::create([
            'actor_id' => $actor?->id,
            'action' => $action,
            'auditable_type' => $auditable?->getMorphClass(),
            'auditable_id' => $auditable?->getKey(),
            'before' => $before,
            'after' => $after,
            'reason' => $reason,
        ]);
    }
}
```

It's injected into controllers via Laravel's constructor auto-resolution (see `ShiftController` and `TillController` below) and called once per meaningful state change — e.g. `shift.open`, `till.open`.

7. API Endpoints

All routes are versioned under `/api/v1` and registered in `routes/api.php`. Public routes need no token; everything else requires a Sanctum bearer token from `/auth/login`.

Method & Path	Auth	Purpose
POST `/api/v1/auth/login`	Public	PIN/password login. Validates active status & store-terminal match, issues a Sanctum token.
POST `/api/v1/auth/logout`	Sanctum	Revokes the current access token.
GET `/api/v1/user`	Sanctum	Returns the authenticated user with roles loaded.
POST `/api/v1/terminals/register`	Public	Pairs a terminal by registration code + store, creating or activating it.
GET `/api/v1/terminals/{terminal}/status`	Public	Ready-status / config check; updates `last_seen_at`.
POST `/api/v1/shifts/open`	Sanctum	Opens a shift for the authenticated user on a terminal; rejects if that terminal already has an active shift.
POST `/api/v1/tills/open`	Sanctum	Opens a till against an open shift with an opening float; requires manager PIN above a configured threshold.

Example — login request/response

```
POST /api/v1/auth/login
{ "login": "cashier@simbisapos.test", "credential": "password", "terminal_id": 1 }

200 OK
{
  "token": "1|vV5pODktbY4kzZT2d7RZgJg3kQ1gYg5DnEIMTl2r262ae9ab",
  "user": {
    "id": 2, "name": "Demo Cashier", "employee_code": "CSH001",
    "store_id": 1, "roles": ["cashier"]
  }
}
```

Example — till open above the manager threshold

```
POST /api/v1/tills/open   { "shift_id": 1, "opening_float": 600 }
422 Unprocessable Entity
{ "message": "Manager verification is required for an opening float of 500 or more.",
  "errors": { "manager_pin": ["Manager verification is required … "] } }

POST /api/v1/tills/open   { "shift_id": 1, "opening_float": 600, "manager_pin": "1234" }
201 Created
{ "till": { "id": 2, "shift_id": 1, "opening_float": "600.0000",
            "status": "open", "manager_verified_by": 1 } }
```

8. Controllers — Code Walkthrough

8.1 AuthController — login

backend/app/Http/Controllers/Api/AuthController.php

```
public function login(Request $request)
{
    $data = $request->validate([
        'login' => ['required', 'string'],
        'credential' => ['required', 'string'],
        'terminal_id' => ['required', 'exists:terminals,id'],
    ]);

    $user = User::query()
        ->where('employee_code', $data['login'])
        ->orWhere('email', $data['login'])
        ->first();

    $terminal = Terminal::findOrFail($data['terminal_id']);

    $credentialValid = $user && (
        Hash::check($data['credential'], $user->password)
        || ($user->pin_hash && Hash::check($data['credential'], $user->pin_hash))
    );

    if (! $user || ! $credentialValid) {
        throw ValidationException::withMessages([
            'login' => ['The provided credentials are incorrect.'],
        ]);
    }

    if (! $user->active) {
        throw ValidationException::withMessages(['login' => ['This account is not active.']]);
    }

    if ($user->store_id !== null && $user->store_id !== $terminal->store_id) {
        throw ValidationException::withMessages([
            'login' => ['This account is not assigned to this store.'],
        ]);
    }

    $token = $user->createToken("terminal-{$terminal->id}")->plainTextToken;

    return response()->json([
        'token' => $token,
        'user' => [
            'id' => $user->id, 'name' => $user->name, 'employee_code' => $user->employee_code,
            'store_id' => $user->store_id, 'roles' => $user->getRoleNames(),
        ],
    ]);
}
```

One identifier field (`login`) accepts either the employee code or email, and one credential field accepts either the password or the PIN — `Hash::check` is tried against both hashed columns. This mirrors the guide's Section 1 login sequence ("Enter PIN, password or staff card") while keeping a single endpoint. The store/terminal check enforces "validate active role and store assignment" server-side, not just in the UI.

8.2 TerminalController — self-registration

`backend/app/Http/Controllers/Api/TerminalController.php`

```
public function register(Request $request)
{
    $data = $request->validate([
        'registration_code' => ['required', 'string'],
        'store_id' => ['required', 'exists:stores,id'],
        'name' => ['nullable', 'string'],
        'app_version' => ['nullable', 'string'],
    ]);

    $terminal = Terminal::firstOrNew([
        'registration_code' => $data['registration_code'],
        'store_id' => $data['store_id'],
    ]);

    $terminal->fill([
        'name' => $data['name'] ?? $terminal->name,
        'app_version' => $data['app_version'] ?? null,
        'status' => TerminalStatus::Active,
        'last_seen_at' => now(),
    ])->save();

    return response()->json(['terminal' => $terminal]);
}
```

`firstOrNew` makes this idempotent by (registration_code, store_id): the same physical terminal calling this again just refreshes its status and last-seen time instead of erroring or duplicating a row.

8.3 ShiftController — open, with duplicate-shift guard

`backend/app/Http/Controllers/Api/ShiftController.php`

```
public function open(Request $request)
{
    $data = $request->validate(['terminal_id' => ['required', 'exists:terminals,id']]);
    $terminal = Terminal::findOrFail($data['terminal_id']);

    $activeShift = Shift::query()
        ->where('terminal_id', $terminal->id)
        ->where('status', ShiftStatus::Open)
        ->first();

    if ($activeShift) {
        throw ValidationException::withMessages([
            'terminal_id' => ['This terminal already has an active cashier shift.'],
        ]);
    }

    $shift = Shift::create([
        'user_id' => $request->user()->id,
        'terminal_id' => $terminal->id,
        'status' => ShiftStatus::Open,
        'opened_at' => now(),
    ]);

    $this->auditLogger->log($request->user(), 'shift.open', $shift, after: $shift->toArray());

    return response()->json(['shift' => $shift], 201);
}
```

Directly implements Section 1's "confirm no active cashier" step of the Open Till workflow — a second cashier cannot open a competing shift on the same terminal while one is already open.

8.4 TillController — open, with manager-threshold verification

`backend/app/Http/Controllers/Api/TillController.php`

```
public function open(Request $request)
{
    $data = $request->validate([
        'shift_id' => ['required', 'exists:shifts,id'],
        'opening_float' => ['required', 'numeric', 'min:0'],
        'manager_pin' => ['nullable', 'string'],
    ]);

    $shift = Shift::findOrFail($data['shift_id']);
    if ($shift->user_id !== $request->user()->id || $shift->status !== ShiftStatus::Open) {
        throw ValidationException::withMessages(['shift_id' => ['This shift is not open for thi
    }

    $manager = null;
    $threshold = config('pos.till_open_manager_threshold');

    if ($data['opening_float'] >= $threshold) {
        if (empty($data['manager_pin'])) {
            throw ValidationException::withMessages([
                'manager_pin' => ["Manager verification is required for an opening float of {$t
            ]);
        }

        $manager = User::role('manager')->get()
            ->first(fn (User $c) => $c->pin_hash && Hash::check($data['manager_pin'], $c->pin_h

        if (! $manager) {
            throw ValidationException::withMessages(['manager_pin' => ['Manager PIN could not b
        }
    }

    $till = Till::create([
        'shift_id' => $shift->id, 'opening_float' => $data['opening_float'],
        'status' => TillStatus::Open, 'manager_verified_by' => $manager?->id,
    ]);

    $till->cashMovements()->create([
        'type' => 'open', 'amount' => $data['opening_float'],
        'reason' => 'Opening float', 'approved_by' => $manager?->id,
    ]);

    $this->auditLogger->log($request->user(), 'till.open', $till, after: $till->toArray());

    return response()->json(['till' => $till], 201);
}
```

Implements Section 1's "manager verifies if required" step: the threshold lives in `config/pos.php` (`POS_TILL_OPEN_MANAGER_THRESHOLD`, default 500), and above it the request must carry a PIN that hashes against a user holding the `manager` role. Every till-open also writes a `cash_movements` row and an audit-log entry.

9. Roles, Permissions & Manager Thresholds

spatie/laravel-permission provides the role/permission tables; roles map to the guide's staff types (cashier, server, supervisor, manager, admin), each with the permissions relevant to Sprint 0.

backend/database/seeders/RolePermissionSeeder.php

```
$permissions = ['shift.open', 'till.open', 'terminal.register', 'manager.override'];
foreach ($permissions as $permission) {
    Permission::firstOrCreate(['name' => $permission]);
}

$roles = [
    'cashier'    => ['shift.open', 'till.open'],
    'server'     => ['shift.open'],
    'supervisor' => ['shift.open', 'till.open', 'manager.override'],
    'manager'    => ['shift.open', 'till.open', 'terminal.register', 'manager.override'],
    'admin'      => $permissions,
];

foreach ($roles as $role => $rolePermissions) {
    Role::firstOrCreate(['name' => $role])->syncPermissions($rolePermissions);
}
```

backend/config/pos.php

```
return [
    'till_open_manager_threshold' => (float) env('POS_TILL_OPEN_MANAGER_THRESHOLD', 500),
];
```

10. Seed Data

`DevDataSeeder` creates one store, one terminal, and two staff accounts so both endpoints and the frontend have something real to talk to during development.

`backend/database/seeders/DevDataSeeder.php (excerpt)`

```
$store = Store::firstOrCreate(['code' => 'STORE-001'], [ ... ]);
$terminal = Terminal::firstOrCreate([ ... ], ['status' => TerminalStatus::Active,  ... ]);

$manager = User::firstOrCreate(['email' => 'manager@simbisapos.test'], [
    'employee_code' => 'MGR001', 'password' => Hash::make('password'),
    'pin_hash' => Hash::make('1234'), 'store_id' => $store->id,
]);
$manager->syncRoles(['manager']);

$cashier = User::firstOrCreate(['email' => 'cashier@simbisapos.test'], [
    'employee_code' => 'CSH001', 'password' => Hash::make('password'),
    'pin_hash' => Hash::make('1111'), 'store_id' => $store->id,
]);
$cashier->syncRoles(['cashier']);
```

Account	Login	Password	PIN	Role
Demo Manager	manager@simbisapos.test / MGR001	password	1234	manager
Demo Cashier	cashier@simbisapos.test / CSH001	password	1111	cashier

11. Automated Tests

PHPUnit feature tests (Laravel's default test runner in this scaffold — not Pest) cover every Sprint-0 endpoint, running against an in-memory SQLite database configured in `phpunit.xml`.

backend/tests/Feature/PosFoundationTest.php (excerpt)

```
public function test_till_open_above_threshold_requires_manager_pin(): void
{
    $cashier = $this->makeCashier();
    $this->makeManager('9999');

    $shiftResponse = $this->actingAs($cashier, 'sanctum')
        ->postJson('/api/v1/shifts/open', ['terminal_id' => $this->terminal->id]);

    $this->actingAs($cashier, 'sanctum')
        ->postJson('/api/v1/tills/open', [
            'shift_id' => $shiftResponse->json('shift.id'), 'opening_float' => 999,
        ])->assertUnprocessable();

    $withPin = $this->actingAs($cashier, 'sanctum')
        ->postJson('/api/v1/tills/open', [
            'shift_id' => $shiftResponse->json('shift.id'),
            'opening_float' => 999, 'manager_pin' => '9999',
        ]);
    $withPin->assertCreated();
    $this->assertNotNull($withPin->json('till.manager_verified_by'));
}
```

Eight tests cover: terminal self-registration, login by password, login by PIN, login rejected on wrong credential, login rejected for an inactive user, shift+till open happy path (asserting the cash-movement and audit-log rows exist), duplicate-shift rejection, and the manager-threshold till-open flow above.

```
$ php artisan test
{"tool":"phpunit","result":"passed","tests":10,"passed":10,"assertions":21,"duration_ms":671}
```

12. How It Was Built — Step by Step

1. **Toolchain check** — confirmed PHP 8.3.31, Composer 2.9.8, Node 24, npm 11, git 2.54 were already installed.
2. **Repo init** — `git init` at the monorepo root.
3. **Laravel install** — `composer create-project laravel/laravel backend` (resolved to Laravel 13.29, the current stable release; the plan's original "Laravel 11" pin was dropped since it's long past support).
4. **API scaffolding** — `php artisan install:api`, which added `routes/api.php`, installed Sanctum, and ran its migration. Newer Laravel skeletons don't ship API routing by default; this command is required first.
5. **Roles/permissions package** — `composer require spatie/laravel-permission`, then `vendor:publish` its config and migration.
6. **Database decision** — discovered this machine has PostgreSQL 17 running but no MySQL, and that installing MySQL needs an elevated (Administrator) shell this session can't grant (a UAC prompt can't be answered non-interactively). Agreed to build against SQLite in the interim; `.env` flips to MySQL with no code changes once it's installed.
7. **Schema** — hand-wrote all 21 migrations (edited the base `users` migration; added stores, terminals, shifts, tills, cash_movements, customers, items, tables, and the seven UUID-keyed sale/payment/credit-note/kitchen-order tables, plus audit_logs). Verified with `php artisan migrate:fresh`.
8. **Enums** — transcribed all six Section-14 state models plus two supporting ones into PHP backed enums.
9. **Models** — one Eloquent model per table with relationships, enum casts, and decimal casts; edited `User` to add Sanctum + Spatie traits.
10. **Audit service** — small `AuditLogger` class so every controller logs consistently.
11. **Config** — added `config/pos.php` for the manager-verification threshold.
12. **Controllers & routes** — `AuthController`, `TerminalController`, `ShiftController`, `TillController`; wired into `routes/api.php` under `/api/v1`, public vs. `auth:sanctum`-gated groups.
13. **Seeders** — `RolePermissionSeeder` and `DevDataSeeder`, called from `DatabaseSeeder`.
14. **Manual verification** — ran `php artisan serve` and exercised every endpoint with `curl` (register terminal → login → open shift → open till below/above threshold → duplicate-shift rejection → terminal status → logout), confirming both the JSON responses and the resulting `audit_logs` rows.
15. **Automated tests** — wrote `PosFoundationTest` covering the same scenarios; full suite passes (10/10).

13. Verification Performed

Check	Result
`php artisan migrate:fresh`	All 21 tables create cleanly, no FK errors
`php artisan migrate:fresh --seed`	Roles/permissions + demo store/terminal/users seed successfully
Manual `curl` pass over every endpoint	All expected 200/201/422 responses observed, matching business rules
Audit trail check via `artisan tinker`	3 audit rows recorded (`shift.open` , 2× `till.open`) matching the manual test session
`php artisan test`	10/10 tests passed, 21 assertions

14. Out of Scope / Next Steps

Everything from Sprint 2 onward in the project's own delivery timeline is deliberately not implemented yet: sale/basket and payment commit logic, discounts/loyalty, receipts, credit-note/refund processing, kitchen routing, tables/tabs, voids and manager overrides beyond the till-open threshold, cash-drawer reconciliation beyond opening, and offline sync. The schema already supports all of it (every table and enum a later sprint needs already exists), so those sprints build endpoints and business logic against structure that's already in place rather than starting from an empty database.